

Unit 1

Lesson 2

CLASS & OBJECT

Objective:

In the following lesson you are able to get an idea of some of the important terminologies used frequently in OOPS such as

- Class
- Object

Now you have an idea what object oriented programming is. Here in this second lecture of OOP we will go through some of the important terminologies used in OOP like class and object in depth.

CLASS

A class is a logical method to organize data and functions in the same structure. They are declared using keyword **class**, whose functionality is similar to that of the C keyword **struct**, but with the possibility of including functions as members, instead of only data.

Its form is:

```
class class_name {  
    permission_label_1:  
        member1;  
    permission_label_2:  
        member2;  
    ...  
} object_name;
```

where ***class_name*** is a name for the class (user defined *type*) and the optional field ***object_name*** is one, or several, valid object identifiers. The body of the declaration can contain ***members***, that can be either data or function declarations, and optionally ***permission labels***, that can be any of these three keywords: **private:**, **public:** or **protected:**. They make reference to the permission which the following ***members*** acquire:

private members of a class are accessible only from other members of their same class or from their *"friend"* classes.

protected members are accessible from members of their same class and *friend* classes, and also from members of their *derived* classes.

Finally, **public** members are accessible from anywhere the class is visible.

If we declare members of a class before including any permission label, the members are considered **private**, since it is the default permission that the members of a class declared with the **class** keyword acquire.

For example:

```
class CRectangle {  
    int x, y;  
    public:  
        void set_values (int,int);  
        int area (void);  
} rect;
```

Declares class **CRectangle** and an object called **rect** of this class (type). This class contains four members: two variables of type **int** (**x** and **y**) in the **private** section (because private is the default permission) and two functions in the **public** section: **set_values()** and **area()**, of which we have only included the prototype.

Notice the difference between class name and object name: In the previous example, **CRectangle** was the class name (i.e., the user-defined type), whereas **rect** was an object of type **CRectangle**. Is the same difference that **int** and **a** have in the following declaration:

```
int a;
```

int is the *class name* (type) and **a** is the *object name* (variable).

On successive instructions in the body of the program we can refer to any of the public members of the object **rect** as if they were normal functions or variables, just by putting the object's name followed by a point and then the class member (like we did with C **structs**).

For example:

```
rect.set_value                                     (3,4);  
myarea = rect.area();
```

but we will not be able to refer to **x** or **y** since they are private members of the class and they could only be referred from other members of that same class. Confused? Here is the complete example of class **CRectangle**:

```
// classes example
```

```
#include <iostream.h>
```

```
class CRectangle {  
    int x, y;  
public:  
    void set_values (int,int);  
    int area (void) {return (x*y);}  
};
```

```
void CRectangle::set_values (int a, int b) {  
    x = a;  
    y = b;  
}
```

```
int main () {  
    CRectangle rect;  
    rect.set_values (3,4);  
    cout << "area: " << rect.area();  
}
```

Output :

area: 12

The new thing in this code is the operator `::` of scope included in the definition of `set_values()`. It is used to declare a member of a class outside it. Notice that we have defined the behavior of function `area()` within the definition of the `CRectangle` class - given its extreme simplicity. Whereas `set_values()` has only its prototype declared within the class but its definition is outside. In this outside declaration we must use the operator of scope `::`.

The scope operator (`::`) specifies the class to which the member being declared belongs, granting exactly the same scope properties as if it was directly defined within the class. For example, in the function `set_values()` of the previous code, we have referred to the variables `x` and `y`, that are members of class `CRectangle` and that are only visible inside it and its members (since they are **private**).

The only difference between defining a class member function completely within its class and to include only the prototype, is that in the first case the function will automatically be considered *inline* by the compiler, while in the second it will be a normal (not-inline) class member function.

The reason why we have made `x` and `y` **private** members (remember that if nothing else is said all members of a class defined with keyword *class* have **private** access) it is because we have already defined a function to introduce those values in the object (`set_values()`) and therefore the rest of the program does not have a way to directly access them. Perhaps in a so simple example as this you do not see a great utility protecting those two variables, but in greater projects it may be very important that values cannot be modified in an unexpected way (unexpected from the point of view of the object).

One of the greater advantages of a class is that we can declare several different objects from it. For example, following with the previous example of class `CRectangle`, we could have declared the object `rectb` in addition to the object `rect` :

```
// class example
```

```
#include <iostream.h>
```

```
class CRectangle {
    int x, y;
public:
    void set_values (int,int);
    int area (void) {return (x*y);}
};
```

```

void CRectangle::set_values (int a, int b) {
    x = a;
    y = b;
}

int main () {
    CRectangle rect, rectb;
    rect.set_values (3,4);
    rectb.set_values (5,6);
    cout << "rect area: " << rect.area() << endl;
    cout << "rectb area: " << rectb.area() << endl;
}

```

Output

rect area: 12

rectb area: 30

Notice that the call to **rect.area()** does not give the same result as the call to **rectb.area()**. This is because each object of class **CRectangle** has its own variables **x** and **y**, and its own functions **set_value()** and **area()**.

On that is based the concept of *object* and *object-oriented programming*. In that data and functions are properties of the object, instead of the usual view of objects as function parameters in structured programming. In this and the following sections we will discuss advantages of this methodology.

In this concrete case, the class (type of object) to which we were talking about is **CRectangle**, of which there are two instances, or objects: **rect** and **rectb**, each one with its own member variables and member functions.

Object

Definition (Object) *An object is an instance of a class. It can be uniquely identified by its name and it defines a state which is represented by the values of its attributes at a particular time.*

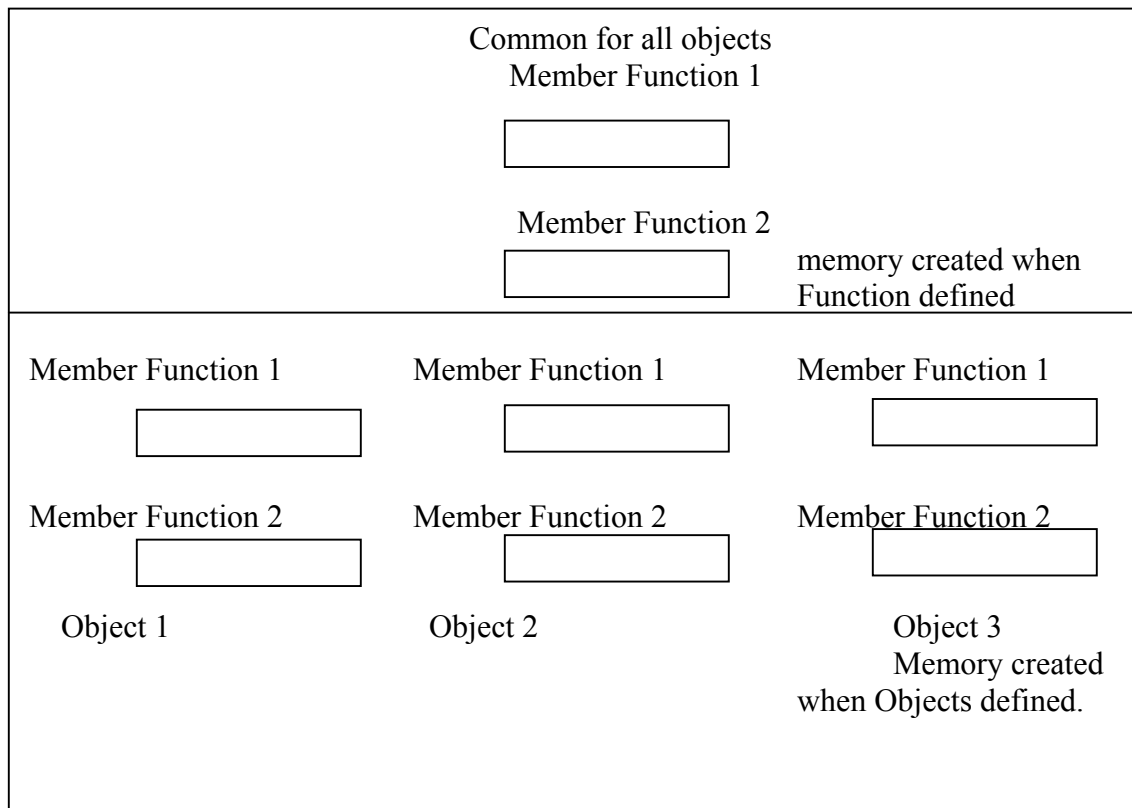
The state of the object changes according to the methods which are applied to it. We refer to these possible sequence of state changes as the *behaviour* of the object:

Definition (Behaviour) *The behaviour of an object is defined by the set of methods which can be applied on it.*

We now have two main concepts of object-orientation introduced, class and object. Object-oriented programming is therefore the implementation of abstract data types or, in more simple words, the writing of classes. At runtime instances of these classes, the objects, achieve the goal of the program by changing their states. Consequently, you can think of your running program as a collection of objects.

MEMORY ALLOCATION FOR OBJECTS

We have stated that the memory space for objects is allocated when they are declared and not when the class is specified. This statement is only partly true. Actually, the member functions are created and placed in the memory space only once when they are defined as a part of a class specification. Since all the objects belonging to that class use the same member functions, no separate space is allocated for member functions when the objects are created. Only space for member variables is allocated separately for each object. Separate memory locations for objects are essential, because the member variables will hold different data values for different objects.



OBJECTS AS FUNCTION ARGUMENTS

Like any other data type, an object may be used as a function argument. This can be done in two ways:

- A copy of the entire object is passed to the function.
- Only the address of the object is transferred to the function.

The first method is called *pass-by-value*. Since a copy of the object is passed to the function, any changes made to the object inside the function do not affect the object used to call the function. The second method is called *pass-by-reference*. When an address of the object is passed to the called function works directly on the actual object used in the call. This means that any changes made to the object inside the function will reflect in the actual object. The pass-by reference method is more efficient since it requires to pass only the address of the object and not the entire object.

Program illustrates the use of objects as function arguments. It performs the addition of time in the hour and minutes format.

Example: Object as argument

```
#include<iostream.h>
```

```
class time
```

```
{
```

```
    int hours;
```

```
    int minutes;
```

```
public:
```

```
    void gettime(int h, int m)
```

```
    {
```

```
        hours = h; minutes = m;
```

```
    }
```

```
    void puttime(void)
```

```

    {
        cout<<hours<<" hours and";
        cout<<minutes<<"minutes"<<"\n";
    }

    void sum(time , time );           /declaration with objects as arguments
};

void time:: sum(time t1, time t2);   //t1 & t2 are objects
{
    minutes = t1.minutes + t2.minutes;
    hours = minutes/60
    minutes = minutes%60;
    hours = hours + t1.hours + t2.hours;
}

int main()
{
    time T1,T2,T3;
    T1.gettime(2,45);               //get T1
    T2.gettime(3,30);               //get T2
    T3.sum(T1,T2);                   //T3=T1+T2
    Cout<< "T1= ";T1.puttime();     //display T1
    Cout<< "T2= ";T2.puttime();     //display T2
    Cout<< "T3= ";T3.puttime();     //display T3
    Return 0;
}

```

Output

T1 = 2 hours and 45 minutes

T2 = 3 hours and 30 minutes

T3 = 6 hours and 15 minutes

Since the member function sum() is invoked by the object T3, with the objects T1 and

T2 as arguments, it can directly access the hours and minutes variables of T3. But, the members of T1 and T2 can be accessed only by using the dot operator (like T1.hours and T1.minutes). Therefore, inside the function sum() the variables hours and minutes refer to T3, T1.hours and T1.minutes refer to T1, and T2.hours and T2.minutes refer to T2.

An object can also be passed as an argument to a non-member function. However, static functions can have access to the public member functions only through the objects passed as arguments to it. These functions cannot have access to the private data members

Attribute

Definition (Attribute) A property of object. It cannot exist independently of the object. Attribute may take other objects as values. It has a type. It describes the range of values that that property may hold.

Method

Definition (Method) A method is associated with a class. An object invokes a method as a reaction to receipt of a message.

A Method is a service that can be requested from any object of the Class to affect behavior. A Method (Operation) can either be a command or a question. A question should never change the state of the object only a command can. The outcome of the Operation depends on the current state of the object.

Summary

- A class is a logical method to organize data and functions in the same structure. They are declared using keyword class
- private members of a class are accessible only from other members of their same class or from their "friend" classes.
- protected members are accessible from members of their same class and friend classes, and also from members of their derived classes.

- public members are accessible from anywhere the class is visible.
- An object is an instance of a class. It can be uniquely identified by its name and it defines a state which is represented by the values of its attributes at a particular time.
- The behaviour of an object is defined by the set of methods which can be applied on it.
- Attribute: A property of object. It cannot exist independently of the object.
- A method is associated with a class. An object invokes a method as a reaction to receipt of a message.

Questions

1. Distinguish between Class and Object
2. Define
 - a) Method
 - b) Attribute.
3. Describe Objects As Function Arguments?